Adaptive Schema Evolution Detection and Mapping Regeneration in ETL Pipelines using Retrieval-Augmented Large Language Models

Raja Karthik Chirumamilla

SRH University of Applied Sciences

School of Technology Leipzig

March 4, 2026

Submitted in partial satisfaction of the requirements for the
Degree of Masters in Computer Science (Big Data and Artificial Intelligence)
in Computer Science (Big Data and Artificial Intelligence)

Supervisor Ghanbarnejad, Fakhteh
Second Supervisor Mohandoss, Smitha

List of Related Publications

This thesis is a collection of schema evolution, schema mapping, ETL automation, application of large language models (LLMs) and retrieval-augmented generation (RAG) in data management. The following publications form the core scientific background for this work:

1. **Mapping adaptation under evolving schemas.** Velegrakis, Y., Miller, R. J., & Popa, L. (2003). *Mapping Adaptation under Evolving Schemas*. In *Proceedings of the 29th International Conference on Very Large Data Bases (VLDB)*, Berlin, Germany.

2. **Semantic adaptation of schema mappings when schemas evolve.** Yu, C., & Popa, L. (2005). *Semantic Adaptation of Schema Mappings when Schemas Evolve*. In *Proceedings of the 31st International Conference on Very Large Data Bases (VLDB)*, Trondheim, Norway.

3. **Survey of schema matching approaches.** Rahm, E., & Bernstein, P. A. (2001). *A Survey of Approaches to Automatic Schema Matching*. *The VLDB Journal*, 10(4), 334–350.

4. **Towards scalable schema mapping using LLMs.** Buss, C., Safari, M., Termehchy, A., Lee, S., & Maier, D. (2025). *Towards Scalable Schema Mapping using Large Language Models*. *arXiv preprint arXiv:2505.24716*.

5. **LLMs and enterprise data integration.** Kayali, M., Wenz, F., Tatbul, N., & Demiralp, Ç. (2024). *Mind the Data Gap: Bridging LLMs to Enterprise Data Integration*. *arXiv preprint arXiv:2412.20331* (CIDR 2025).

6. **Retrieval-enhanced schema matching with LLMs.** Sheetrit, E., Brief, M., Mishaeli,

M., & Elisha, O. (2024). *ReMatch: Retrieval Enhanced Schema Matching with LLMs*. *arXiv preprint arXiv:2403.01567*.

7. **Retrieval-augmented generation.** Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., *et al.* (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.

8. **LLMs trained on code.** Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., *et al.* (2021). *Evaluating Large Language Models Trained on Code*. *arXiv preprint arXiv:2107.03374*.

9. **Foundation models for data wrangling.** Narayan, A., Chami, I., Orr, L., & Ré, C. (2022). *Can Foundation Models Wrangle Your Data? Proceedings of the VLDB Endowment*, 16(4), 738–746.

The need for an adaptive, evidence-grounded approach for schema evolution detection and mapping regeneration in ETL pipelines was motivated by these works. They also provide the necessary baselines and design patterns against which the contributions of this thesis can be evaluated.

Acknowledgements

I would like to express my sincere gratitude to my supervisor(s) for their guidance, feedback, and support throughout the development of this thesis. Their expertise in data engineering and applied artificial intelligence helped shape the research direction and strengthened the scientific rigor of the work.

I am also grateful to the faculty and teaching staff of my program for providing the methodological foundation in research design, academic writing, and data engineering practices that informed this project. I would like to thank my peers for valuable discussions and for sharing perspectives that improved the clarity of the problem formulation and evaluation strategy.

Finally, I would like to thank my family and friends for their encouragement and patience during the thesis process. Their support made it possible to stay focused and complete this work.

Abstract

Modern Extract–Transform–Load (ETL) pipelines are frequently invalidated by upstream schema evolution, leading to high manual maintenance costs and data downtime. This thesis addresses the central research question: To what extent can Retrieval-Augmented Large Language Models (RAG-LLMs) automate the detection of schema changes and the subsequent regeneration of valid transformation logic? To answer this, a three-stage methodology was developed and implemented: (i) a RAG-based LLM-assisted schema change detection mechanism, (ii) a RAG-based retrieval strategy that assembles task-relevant evidence from historical schemas and validated mappings, and (iii) an LLM-assisted regeneration component. The approach was evaluated using the Open Source NYC TLC Trip Record data across multiple evolution scenarios.

The findings reveal that suggested workflow achieved a cell-level mapping agreement of 98% and decreased the time-to-adaptation from hours of manual engineering to an average of 86.5 seconds. The thesis also revealed some limitations like the stochastic character of smaller models like Mistral and unable to address column deletions accurately. The thesis shows that semantic integrity still requires a "human-in-the-loop" inspection although RAG-LLMs speed up ET maintenance. This work offers a scalable architecture for self-adapting data pipelines.

List of Tables

4.1 Schema changes for NYC taxi V1 $\rightarrow$ V2. 32

5.1 Schema change detection metrics for canonical $\leftrightarrow$ V2 (from `results/canonical_changes_v2`
5.2 Schema detection F_1 scores for heuristic vs. RAG-LLM detectors on the canonical $\leftrightarrow$ V2 scenario. Both methods are evaluated against the canonical change set in `results/canonical_changes_v2.json`. 39
5.3 Ablation study: zero-shot vs. RAG-based canonical mappings for NYC taxi V2 across two LLM backends. 43
5.4 Summary of thesis-wide evaluation metrics across schema detection, canonical ingestion, and end-to-end AI-assisted execution. 46

List of Figures

4.1 High-level architecture of the prototype system. The process flows from schema extraction to RAG-based code regeneration, culminating in data loading via the ETL pipeline into SQLite. 24

5.1 Schema change detection result for the canonical $\leftrightarrow$ V2 scenario. The heuristic detector identifies added canonical columns, removed V2-only columns, and the `trip_km` $\rightarrow$ `trip_distance` rename, matching the ground-truth change set used for canonical ingestion. 39

5.2 Comparison of baseline vs. LLM-generated canonical ingestion for NYC taxi V2. Cell agreement (%), row-count match, and per-column agreement are shown for `llama3`, `mistral`, and the Hugging Face `gpt-oss-120b` model. The baseline (deterministic mapping) serves as the reference; the HF model attains the highest cell-level agreement. 42

5.3 Ingestion success rate over multiple runs per model for the V2 $\rightarrow$ canonical scenario. Success indicates that the generated transform executed without error and produced a row count matching the baseline. The figure illustrates the variability of `llama3` and the more consistent success of the larger hosted model across runs. 42

5.4 End-to-end AI-assisted workflow: success rate and duration per model for the NYC taxi V1 $\rightarrow$ V2 scenario. All models achieve full success under the automated DB checks; the bar chart highlights the large difference in wall-clock time between local (`llama3`, `mistral`) and hosted (`gpt-oss-120b`) backends. 45

Table of Contents

1 Introduction **1**

1.1 Introduction . . . 1

1.2 Problem Statement . . . 2

1.3 Research Aim and Objectives . . . 3

1.3.1 Research Aim . . . 3

1.3.2 Objectives . . . 3

1.4 Research Questions . . . 4

1.4.1 Main Research Question . . . 4

1.4.2 Sub-Questions . . . 4

1.5 Scope and Delimitations . . . 5

1.6 Outline of Thesis Structure . . . 5

2 Basics and Theory **7**

2.1 Basics and Theory . . . 7

2.1.1 Schema Evolution . . . 7

2.1.2 Schema Matching and Mapping . . . 7

Schema mappings as functions . . . 8

Information loss and compatibility . . . 8

2.1.3 ETL Pipelines and Orchestration . . . 9

2.1.4 Large Language Models and Retrieval-Augmented Generation . . . 9

Parametric and non-parametric knowledge . . . 9

2.2 Dataset Used in This Thesis . . . 10

3 Related Work **12**

3.1 Introduction . . . 12

3.2 Schema Evolution and Mapping Adaptation . . . 12

3.2.1 Foundational Mapping Adaptation Frameworks . . . 12

3.2.2 Schema Evolution in Practice . . . 13

3.2.3 Commercial Schema Evolution Support . . . 13

3.3 Schema Matching and Discovery . . . 14

3.3.1 Traditional Schema Matching Approaches 15
3.3.2 LLM-Based Schema Matching 15
3.3.3 Knowledge Graph-Enhanced Schema Matching 16
3.4 Retrieval-Augmented Generation for Data Integration 16
3.4.1 Foundational RAG Framework 17
3.4.2 RAG for Tabular Data Discovery 17
3.5 Large Language Models for ETL and Data Engineering 17
3.5.1 Code Generation with LLMs 17
3.5.2 Autonomous ETL Pipeline Design 18
3.6 Adaptive Schema Evolution Detection 19
3.6.1 Machine Learning-Based Detection 19
3.6.2 Text-to-SQL Robustness Against Schema Evolution 19
3.7 Gaps and Research Opportunities 20
3.8 Summary 21

4 Method **22**
4.1 Overall Research Design 22
4.2 System Architecture 23
4.2.1 Data and storage layer 23
4.2.2 ETL pipeline 24
4.2.3 Schema change detection module 25
4.2.4 AI mapping regenerator 27
Retrieval-augmented generation (RAG) workflow 28
4.3 Dataset and Schema Evolution Scenario 31
4.3.1 NYC yellow taxi dataset 31
4.4 Taxi Transform Logic 32
4.5 Evaluation Protocol 33
4.5.1 Scenarios 33
4.5.2 Conditions 34
4.5.3 Metrics 34
4.5.4 Ablation Study: Zero-Shot vs. RAG-Grounded Generation 35

5 Results **37**
5.1 Schema Change Detection Results (RQ1) 37
5.1.1 NYC Taxi V1 $\rightarrow$ V2 37
5.1.2 Canonical $\leftrightarrow$ V2 38
5.1.3 Heuristic vs. RAG-LLM Detection for Canonical $\leftrightarrow$ V2 38
5.2 Canonical Ingestion Results (RQ2) 40

5.2.1 Deterministic Baseline 40
5.2.2 LLM-Generated Canonical Mapping: `llama3` 40
5.2.3 LLM-Generated Canonical Mapping: `mistral` 41
5.3 Ablation Results: Zero-Shot vs. RAG-Grounded Mapping 41
5.4 End-to-End AI-Assisted Workflow Results (RQ3) 44
5.4.1 Scenario: NYC Taxi V1 $\rightarrow$ V2 44
5.5 Summary of Key Quantitative Metrics 45

6 Discussion **47**
6.1 Overview 47
6.2 Discussion of RQ1: Schema Change Detection 47
6.3 Discussion of RQ2: Canonical Ingestion and Mapping Regeneration 48
6.3.1 Effect of Retrieval vs. Zero-Shot Prompting 49
6.4 Discussion of RQ3: Operational Feasibility and Effort Reduction 50
6.5 Human-in-the-Loop and Automation Bias 51
6.6 Limitations and Threats to Validity 52
6.6.1 Dataset and Scenario Scope 52
6.6.2 Model and Prompt Dependence 53
6.6.3 Operational and Organisational Factors 53
6.7 Relation to Commercial Tools and Practical Implications 53
6.8 Summary 54

7 Conclusions **55**
7.1 Conclusions 55
7.1.1 Summary of Contributions 55
7.1.2 Answers to the Research Questions 56
7.1.3 Outlook and Future Work 56

Chapter 1

Introduction

1.1 Introduction

Organizations increasingly rely on data-driven decision making, hence operational systems like transactional databases, APIs and SaaS applications are continuously integrated into analytics platforms. Extract-Transform-Load(ETL) is a common way to do so, ETL pipelines ingest, clean, reconcile and publish data for downstream reporting, data analytics and machine learning. However, real-world data sources evolve schemas when requirements change, upstream systems are upgraded, or new attributes are introduced. Even small schema changes like renaming a column will invalidate existing transformations and break downstream dependencies.

The academic literature has long recognized schema evolution as a fundamental challenge in data integration. Early work on mapping adaptation studied how schema mappings (declarative specifications that relate a source schema to a target schema) should be maintained as schemas change [19, 20]. These mapping-centric approaches are central to many integration paradigms and have been closely connected to schema matching research, which aims to identify correspondences between heterogeneous schemas [16]. Despite decades of progress, practitioners still face substantial costs when pipelines break and often requires manual diagnosis, re-implementation of transformations, and extensive validation.

Recent advances in large language models (LLMs) have demonstrated strong capabilities in code generation and program understanding [3]. Retrieval-augmented generation (RAG) further improves reliability by grounding generation in external evidence (e.g., doc-

umentation, examples, previously validated mappings) [11]. In data management, emerging research suggests that foundation models can support data wrangling tasks, but robust end-to-end methods for schema evolution in production ETL remain an open challenge [15].

The thesis involves the following research streams: (i) schema evolution and mapping adaptation, (ii) ETL pipeline operation and maintenance, and (iii) LLM/RAG-based automation for generating transformation logic. The main objective is to reduce the manual effort required to keep ETL pipelines correct and operational as schemas evolve, while preserving the semantics embedded in the existing, validated mappings.

1.2 Problem Statement

ETL pipelines typically encode transformation logic that assumes a specific schema for the source and target datasets. When a schema evolves, this logic can become partially or fully invalid. Prior research provides foundational methods for *adapting* mappings under schema evolution. Velegrakis et al. proposed an incremental framework that detects affected mappings and generates rewritings consistent with schema semantics and constraints [19]. Yu and Popa studied a composition-based approach, showing that mapping composition can better preserve semantics across complex evolution scenarios and that pruning is critical for scalability [20]. While these contributions are substantial, they also highlight unresolved gaps that are particularly relevant for modern ETL practice:

- **Change detection and classification**: Classical approaches often assume that schema evolution (or a sequence of primitive changes) is already known. In many operational environments, schema changes are discovered only after failures occur.
- **Automation of mapping regeneration**: Rule-based adaptation can be brittle or labor-intensive when evolution is complex or poorly documented. Generating maintainable transformation code still commonly requires human expertise.
- **Operational integration**: Mapping adaptation research has historically been evaluated in isolation from workflow orchestration and deployment constraints. Modern

ETL systems require integration with scheduling, monitoring, and rollback mechanisms.

- **Semantic preservation**: A key requirement is to preserve the *intended meaning* of existing mappings, not merely achieve syntactic validity. This is difficult when multiple alternative rewritings are plausible.

Therefore, the main problem addressed in this thesis is to *Detect schema evolution events and regenerate mapping transformations in ETL pipelines in a way that preserves semantics and is operationally feasible, using evidence-grounded LLM techniques*. In particular, the work investigates constructing a retrieval corpus from historical schema versions, machine-readable change sets, and previously validated transformation code, and use this evidence to drive the regeneration of executable ETL mappings under schema evolution.

1.3 Research Aim and Objectives

1.3.1 Research Aim

The aim of this thesis is to design and evaluate an adaptive approach for schema evolution detection and mapping regeneration in ETL pipelines, leveraging retrieval-augmented large language models to produce executable, semantically grounded transformation updates.

1.3.2 Objectives

1. **Design a schema evolution detection mechanism** that identifies and classifies schema changes relevant to ETL transformations (e.g., additions, deletions, renames, type changes, and structural changes).
2. **Develop a retrieval strategy** that gathers relevant evidence for regeneration (e.g., historical mappings, schema documentation, prior transformations, and example instances) to ground generation and reduce hallucination [11].

3. **Implement an LLM-assisted mapping regeneration component** that generates candidate transformation updates informed by retrieved evidence [3].

4. **Evaluate the approach** in terms of detection accuracy, mapping correctness, and operational performance, and compare against baselines from mapping adaptation literature [19, 20].

1.4 Research Questions

1.4.1 Main Research Question

RQ0: How can retrieval-augmented large language models with detected schema evolution be applied to automatically regenerate mapping transformations in ETL pipelines while preserving data semantics and maintaining operational reliability?

1.4.2 Sub-Questions

- **RQ1 (Detection):** What precision, recall, and F_1 scores can the schema change detector achieve for added, removed, and renamed columns in the NYC taxi V1 $\rightarrow$ V2 evolution scenario when evaluated against ground-truth change sets?

- **RQ2 (Regeneration):** For the NYC taxi scenarios, what cell-level agreement (accuracy), null-mismatch counts, and per-column agreement do RAG-grounded LLM-generated canonical mappings achieve relative to a deterministic baseline, and how do these metrics vary across different LLMs?

- **RQ3 (Operational feasibility):** In the orchestrated ETL workflow, what success rate and time-to-adaptation (including the time reduction ratio between manual and AI-assisted conditions) are achieved when integrating detection and regeneration components, and how do these compare with purely manual repair?

1.5 Scope and Delimitations

This thesis focuses on schema evolution in the context of structured data pipelines where transformations can be expressed in declarative or programmatic forms (e.g., SQL and Python/pandas). The scope includes:

- Detection and classification of common evolution events (add/remove/rename/type change/structural shifts).
- Retrieval of historical artifacts (e.g., prior schema versions, mapping specifications, transformation scripts, and example instances) to ground regeneration.
- Generation of candidate transformation updates and validation via automated checks (e.g., schema conformity and basic data-quality checks).

The thesis explicitly **does not** aim to solve all possible forms of schema evolution across arbitrary semi-structured systems. In particular, it does not attempt full ontology alignment, complete automation without human review in safety-critical deployments, or universal support for every data model. These delimitations keep the research tractable and enable rigorous evaluation within the available time and resources.

1.6 Outline of Thesis Structure

Chapter 1 Introduces the research context, problem, aims, and questions.

Chapter 2 presents the basics and theoretical foundations, including schema evolution, schema matching and mapping, ETL orchestration, and retrieval-augmented language models.

Chapter 3 reviews related work on schema evolution, mapping adaptation, schema-centric use of LLMs, and adaptive detection.

Chapter 4 describes the methodological approach, including the system architecture, datasets, and evaluation design.

Chapter 5 reports experimental results for schema change detection, canonical ingestion, and end-to-end AI-assisted workflows.

Chapter 6 discusses these results in relation to the literature, articulates limitations, and draws out practical implications. Finally, Chapter 7 concludes the thesis with a concise summary of contributions and an outlook on future work.

Chapter 2

Basics and Theory

2.1 Basics and Theory

This chapter introduces the core concepts required to understand the thesis. The chapter includes schema evolution, schema matching and mapping, ETL pipeline operation, and retrieval-augmented generation (RAG) with large language models (LLMs). While this chapter provides foundational definitions and terminology used throughout the thesis, The detailed literature survey is presented in Chapter 3 (Related Work).

2.1.1 Schema Evolution

Schema evolution, in general, refers to the process of modifying the schema of a database system in a way that preserves existing data and maintains its compatibility with the old schema. This concept is crucial in the context of large-scale data management, where it is often necessary to adapt database schemas over time to meet changing business requirements, without causing disruption to ongoing operations.

2.1.2 Schema Matching and Mapping

Schema matching Schema matching is the process of developing semantic matches between two or more schemas. The purpose of schema matching is generally either to merge two or more databases, or to enable queries on multiple, heterogeneous databases to be formulated on a single schema [6]. **Schema mappings** involves mapping the fields

or attributes of one dataset to the fields or attributes of another dataset, ensuring that the data types, formats, and other properties match between the two datasets.

Schema mappings as functions

Conceptually, a schema mapping can be viewed as a (partial) function $f : S \to C$ from a source schema S to a canonical or target schema C. Each attribute in C is defined in terms of one or more attributes in S, potentially with additional business logic (for example, computing `trip_revenue` from multiple monetary fields). When a schema evolves from S to S', the goal of adaptation is to obtain a new mapping $f' : S' \to C$ that preserves the intended semantics of f while remaining well-defined under the new structure.

Formally-oriented work in mapping adaptation [19, 20] uses algebraic and composition-based reasoning to relate S, S', and mappings between them. Although this thesis does not rely on full categorical or logical formalisms, it adopts the same intuition: mappings are structure-preserving transformations, and evolution is captured by explicit change operations that must be propagated through these transformations.

Information loss and compatibility

Schema evolution can be characterised in terms of potential information loss. Additive changes (such as adding new attributes) are often lossless with respect to existing mappings, whereas deletions or coarse-graining can be lossy. In practice, this means that some evolution events can be accommodated by extending a mapping, while others require re-defining target attributes or accepting partial loss.

In this thesis, the canonical schema is designed as a stable target C, and evolution in the NYC taxi dataset is modelled as changes in the source schema S (for example, renaming `trip_distance` to `trip_km`). The schema change detector explicitly identifies added, removed, and renamed attributes, and the mapping regenerator uses this information to construct f'. Where attributes are removed in the source, the corresponding parts of f cannot be preserved; the evaluation therefore focuses on equality for overlapping attributes between baseline and regenerated mappings.

2.1.3 ETL Pipelines and Orchestration

ETL pipelines operationalize data movement and transformation into analytical systems. In production, these pipelines are typically orchestrated as workflows with dependencies, retries, monitoring, and scheduling. While orchestration improves reliability of execution, it does not remove the underlying challenge of maintaining transformation correctness under schema evolution.

2.1.4 Large Language Models and Retrieval-Augmented Generation

Recent LLMs have demonstrated strong ability to generate and refactor code [3]. However, pure generation can hallucinate or omit important constraints. Retrieval-augmented generation (RAG) addresses this by retrieving relevant external context (e.g., documentation, examples, historical mappings) and conditioning generation on that evidence [11]. This thesis applies RAG to ground mapping regeneration in previously validated artifacts and schema-evolution examples.

Parametric and non-parametric knowledge

In the RAG framework [11], a distinction is made between *parametric* knowledge (what is encoded in the model weights) and *non-parametric* knowledge (external artefacts retrieved at inference time). Parametric knowledge includes general programming ability, statistical regularities in code, and broad intuitions about tabular data. Non-parametric knowledge, by contrast, consists of concrete schemas, change descriptions, and code snippets specific to a given ETL pipeline.

The mapping regenerator in this thesis follows this pattern: it retrieves JSON-serialised schemas for old and new versions, structured change sets produced by the detector, and snippets of the current transformation logic. These artefacts constrain the LLM so that it operates as a specialised code synthesiser that must respect a fixed input/output interface and a canonical target schema, rather than inventing transformations from scratch. This separation of concerns allows a single general-purpose model to adapt to different domains as long as appropriate schema and change artefacts are provided.

2.2 Dataset Used in This Thesis

The concepts introduced in the Basics and Theory chapter are instantiated in this thesis using a taxi trip dataset inspired by the New York City Taxi and Limousine Commission (TLC) data. Instead of directly using full production-scale files, we construct two controlled source versions of the dataset in Python together with a single canonical target schema to capture typical schema evolution and mapping patterns. All versions contain attributes such as pickup and dropoff times, trip distance, passenger counts, vendor identifiers, and payment-related fields, but with deliberate differences between the source versions and the canonical target.

In the first source version (`v1`), the schema reflects an early design with identifiers for vehicles and drivers, timestamps, coordinates, and basic trip metrics like `trip_time_in_secs` and `trip_distance`. The second source version (`v2`) introduces column renames `trip_time_in_secs` is replaced by `trip_duration`) and adds new payment and fare-related attributes. In addition, we define a single-table canonical schema that all source versions can be mapped into; this canonical schema normalises naming conventions, collects business-relevant measures such as trip duration and revenue, and provides a stable target for both the deterministic baseline and LLM-generated mappings. These artefacts are generated and analyzed in the Jupyter notebook `nyc_taxi_dataset_analysis.ipynb`.

For each source version and for the canonical target, we extract a structured schema description from the corresponding pandas DataFrame, including column names, data types, nullability, uniqueness statistics, and example values. We then compare schema pairs (`v1` $\rightarrow$ `v2` and each source version $\rightarrow$ the canonical schema) to identify added and removed columns, potential renames, and type changes. The resulting schemas and change sets are saved as JSON files in the `schemas/` directory and serve as concrete inputs for the schema evolution detection and canonical mapping regeneration components implemented elsewhere in the project.

In addition to schema-level analysis, we calculate fundamental data properties including dataset shapes, missing values, duplicate rows, and summary statistics for attributes in

addition to schema-level analysis. They support reasoning about the effects of schema modifications on ETL logic and verifies whether the synthetic data behaves realistically. In the ETL pipeline, RAG-enhanced LLMs create and modify transformation code based on previous mappings and schema versions using the same extracted schemas and recognized modifications.

Chapter 3

Related Work

3.1 Introduction

This chapter positions the thesis within existing work on schema evolution, schema matching, retrieval-augmented generation (RAG), and the use of large language models (LLMs) for data integration and ETL. Rather than providing an exhaustive survey, the focus is on a set of representative contributions that directly inform the design choices in this project and motivate the research questions.

3.2 Schema Evolution and Mapping Adaptation

The problem of maintaining data mappings as schemas evolve has been studied extensively in the database and data integration literature. Early work established both theoretical and practical foundations for mapping adaptation.

3.2.1 Foundational Mapping Adaptation Frameworks

Velegrakis et al. [19] introduced the ToMAS (Toronto Mapping Adaptation System) framework, which automatically detects and adapts affected mappings when schemas evolve. Their approach handles both relational and XML schemas, including complex structures with nested constraints. The system generates all rewritings consistent with schema semantics while maintaining user intent embodied in existing mappings. A key contribution is the ability to handle cascading changes affecting multiple schema components, not just local modifications. The framework supports changes to either source or target

schemas and can preserve semantic relationships even when mappings are syntactically unchanged.

Building on this work, Yu and Popa [20] proposed a composition-based approach for semantic adaptation of schema mappings. They demonstrated that mapping composition can better capture the semantics of both original mappings and schema evolution compared to incremental approaches. Their system includes mapping pruning techniques that significantly accelerate the adaptation process, making it practical for large-scale scenarios. The work provides comprehensive experimental analysis showing the practical applicability of the composition approach across various schema evolution scenarios.

Rahm and Bernstein [16] provided a seminal survey of automatic schema matching approaches, establishing a taxonomy that has influenced subsequent research. Their work addresses how schema matching is critical for applications including XML message mapping, data warehouse loading, and schema integration. The survey categorizes matching techniques into instance-based, schema-based, and hybrid approaches, providing a foundation for understanding the relationship between schema matching and mapping adaptation.

3.2.2 Schema Evolution in Practice

Research on schema evolution has expanded beyond theoretical frameworks to address real-world challenges. Empirical studies have shown that database schemas evolve in predictable patterns, with long periods of stability interrupted by bursts of maintenance activity [4]. Modern database systems have incorporated online schema evolution capabilities, but the challenge of maintaining dependent applications and ETL pipelines remains significant.

3.2.3 Commercial Schema Evolution Support

Modern cloud data platforms such as Snowflake, BigQuery, and Databricks provide built-in support for schema evolution at the storage and ingestion layers. Typical features include the ability to ingest data with additional fields without failing a load job, to apply schema-on-read over semi-structured representations (e.g. JSON or variant types), and

to evolve table definitions over time using `ALTER TABLE` statements. These mechanisms are extremely useful in practice because they reduce the operational friction of handling unexpected fields or minor structural changes in upstream data sources.

Beyond the warehouse itself, higher-level data transformation and integration tools provide additional support. Frameworks such as dbt (Data Build Tool) enable versioned SQL models, snapshots for slowly changing dimensions, and incremental materialisations, while extract–load platforms such as Fivetran automatically adjust destination schemas when new source fields appear. Managed ETL services such as AWS Glue offer schema registries and compatibility checks for streaming and batch consumers.

These tools substantially improve operational robustness and developer productivity, but they still rely on humans to adapt custom transformation logic when schemas change. For example, when a column is renamed or split, a dbt model must be edited manually; when a connector adds a field, downstream business logic in SQL or Python is not automatically updated; when a schema registry flags an incompatibility, an engineer must repair the corresponding jobs.

The focus of this thesis is therefore complementary to these capabilities. We explicitly detect and classify schema evolution events between source versions, persist them as structured JSON artefacts, and use this information to drive retrieval-augmented LLM-based regeneration of ETL mappings. Rather than aiming to replace warehouse-level schema evolution or commercial ETL platforms, the goal is to show how LLMs can be integrated into the transformation layer to reduce manual effort in updating mapping code while maintaining existing business logic.

3.3 Schema Matching and Discovery

Schema matching—the task of identifying correspondences between heterogeneous schemas—is a fundamental prerequisite for mapping generation and adaptation. Recent advances have leveraged machine learning and language models to improve matching accuracy and reduce manual effort.

3.3.1 Traditional Schema Matching Approaches

Traditional schema matching methods rely on linguistic similarity (e.g. string matching, tokenisation), structural similarity (e.g. graph-based comparisons), and instance-based techniques (e.g. data value comparisons). These approaches have been extensively surveyed and evaluated [16], but they often struggle with semantic ambiguities and domain-specific knowledge gaps.

3.3.2 LLM-Based Schema Matching

Recent work has demonstrated that large language models can effectively bootstrap schema matching using only element names and descriptions, without requiring labelled training data [12]. This capability is particularly valuable in enterprise settings where obtaining labelled data is expensive and time-consuming.

SCHEMORA [8] combines LLMs with hybrid retrieval techniques in a prompt-based approach, enabling efficient candidate matching without exhaustive pairwise comparisons. By enriching schema metadata and leveraging both vector-based and lexical retrieval, SCHEMORA improves matching accuracy and scalability. Evaluated on the MIMIC-OMOP benchmark, it achieves state-of-the-art performance with gains of 7.49% in HitRate@5 and 3.75% in HitRate@3 over previous best results. The work provides practical guidance on model selection and underscores the critical role of retrieval in LLM-based schema matching.

Magneto [13] addresses cost and accuracy trade-offs by combining small language models (SLMs) with LLMs in a two-phase pipeline: retrieval using efficient SLM-based strategies, followed by LLM-based reranking. The method includes self-supervised fine-tuning using LLM-generated synthetic training data and effective prompting strategies for reranking. Magneto achieves high accuracy across different domains while maintaining scalability, and introduces a new benchmark with real biomedical datasets.

LLMatch [12] is a unified, modular framework that decomposes schema matching into three stages: schema preparation, table-candidate selection, and column-level alignment. It introduces a Rollup-Drilldown optimisation strategy for multi-table schema matching

and includes SchemaNet, a benchmark for complex real-world schema pairs. The framework significantly improves matching accuracy in complex schema matching settings and substantially boosts engineer productivity in real-world data integration.

Matchmaker [17] uses compositional language model programmes with zero-shot self-improvement through synthetic in-context demonstrations, avoiding the need for labelled data. The system comprises three main components: candidate generation, refinement, and confidence scoring. When tested on real-world medical schema matching benchmarks, Matchmaker outperformed previous ML-based approaches by approximately 20%, demonstrating effectiveness for accelerating data integration.

3.3.3 Knowledge Graph-Enhanced Schema Matching

KG-RAG4SM [14] introduces a knowledge-graph-based RAG model for schema matching. The approach uses vector-based, graph-traversal-based, and query-based graph retrievals, as well as a hybrid approach and ranking schemes that identify the most relevant subgraphs from external large knowledge graphs. The work demonstrates that KG-based retrieval-augmented LLMs can generate more accurate results for complex matching cases without any retraining. Experimental results show that KG-RAG4SM outperforms LLM-based state-of-the-art methods by 35.89% and 30.50% in terms of precision and F1 score on the MIMIC dataset, respectively, and outperforms pre-trained language-model-based methods by 69.20% and 21.97% in precision and F1 score on the Synthea dataset. The results also demonstrate that the approach is more efficient in end-to-end schema matching and scales to retrieve from large knowledge graphs.

3.4 Retrieval-Augmented Generation for Data Integration

Retrieval-augmented generation (RAG) has emerged as a powerful paradigm for improving the reliability and accuracy of language models by grounding generation in external evidence.

3.4.1 Foundational RAG Framework

Lewis et al. [11] introduced RAG models, which combine pre-trained parametric memory (a seq2seq model) with non-parametric memory (a dense vector index of Wikipedia accessed via a pre-trained neural retriever). The authors propose two RAG formulations: one that conditions on the same retrieved passages throughout generation, and another that can use different passages per token. They evaluate their approach on knowledge-intensive NLP tasks and achieve state-of-the-art results on three open-domain question answering tasks, surpassing both parametric seq2seq models and task-specific retrieve-and-extract architectures. This foundational work demonstrates how RAG can address limitations in how language models access and manipulate knowledge.

3.4.2 RAG for Tabular Data Discovery

Pneuma [1] is a retrieval-augmented generation system designed for discovering and preparing tabular data using large language models. The system leverages LLMs for both table representation and retrieval, combining them with traditional information retrieval techniques like full-text and vector search. Pneuma preserves schema and row-level information while augmenting LLMs with traditional IR techniques. Evaluation on six real-world datasets (enterprise data, scientific databases, warehousing data, and open data) shows that Pneuma outperforms widely used table search systems and state-of-the-art RAG systems in accuracy and resource efficiency.

3.5 Large Language Models for ETL and Data Engineering

The application of large language models to data engineering tasks has gained significant attention, with particular focus on automating ETL pipeline design and code generation.

3.5.1 Code Generation with LLMs

Chen et al. [3] introduced Codex, a GPT language model fine-tuned on publicly available code from GitHub. The paper evaluated Codex's Python code-writing capabilities using HumanEval, a benchmark for measuring functional correctness. Codex solved 28.8% of

HumanEval problems, compared to 0% for GPT-3 and 11.4% for GPT-J. The investigation revealed that sampling multiple solutions dramatically improved results, solving 70.2% of problems with 100 samples per problem. This work demonstrates the potential of LLMs for code generation tasks, which is directly relevant to generating ETL transformation code.

Narayan et al. [15] explored whether foundation models can wrangle data, examining their capabilities for data integration and transformation tasks. The work provides insights into how LLMs can support data management operations, though robust end-to-end methods for production ETL remain an open challenge.

3.5.2 Autonomous ETL Pipeline Design

FlowETL [5] presents an autonomous example-driven ETL pipeline architecture designed to automatically standardise and prepare input datasets according to a concise, user-defined target dataset. FlowETL is an ecosystem of components including a planning engine that uses paired input–output dataset samples to construct a transformation plan, an ETL worker that applies the plan, and monitoring and logging for observability. The results show promising generalisation capabilities across 14 datasets of various domains, file structures, and file sizes. This work addresses a major limitation of modern ETL solutions: the extensive need for human-in-the-loop involvement in designing context-specific transformations.

Recent work on autonomous ETL pipelines [18] demonstrates that generative AI can analyse requirements to build pipeline logic, construct schema mappings, and create infrastructure-as-code templates for deployment. These systems support continuous validation of transformation logic, dynamic optimisation, and self-healing capabilities through reinforcement-learning feedback loops. Platforms like Microsoft Fabric and Databricks integrate AI copilots that enable both initial code generation and ongoing pipeline management.

3.6 Adaptive Schema Evolution Detection

Detecting schema evolution events is a prerequisite for adaptive mapping regeneration. Recent work has applied machine learning and deep learning techniques to improve detection accuracy and reduce false alarms.

3.6.1 Machine Learning-Based Detection

AutoSchema [10] is a self-supervised learning framework that detects and adapts to schema drift in real-time data streams. The framework uses two key components: a drift detection system employing contrastive learning, which reduces false alarms by 22% compared to rule-based methods, and a dynamic schema adapter using graph-based metadata learning to rebuild schema mappings automatically. The approach achieves 98.3% accuracy in detecting schema drift and enables adaptation in under one second. Unlike static schema management approaches or manual interventions, AutoSchema addresses the streaming context where data must be processed on the fly without pipeline interruption.

SAVeD (Semantic Aware Version Discovery) [7] uses contrastive learning and transformer encoders to identify versions of structured datasets without metadata or labels. The framework employs a modified SimCLR pipeline with augmented table views to learn semantic similarity, demonstrating competitive performance on benchmark datasets. SAVeD addresses the common challenge in data science where datasets undergo various transformations that create multiple versions of the same underlying data.

3.6.2 Text-to-SQL Robustness Against Schema Evolution

EvoSchema [21] is a comprehensive benchmark designed to assess and enhance the robustness of text-to-SQL systems when database schemas evolve. The work introduces a novel schema evolution taxonomy encompassing ten perturbation types across column-level and table-level modifications, systematically simulating real-world database schema changes. Evaluation of both open-source and closed-source large language models revealed that table-level perturbations significantly impact model performance more than column-level changes. The work demonstrates that training models on diverse schema

designs can improve robustness by forcing models to distinguish schema differences for the same questions, avoiding spurious pattern learning.

3.7 Gaps and Research Opportunities

While the reviewed literature provides substantial foundations, several gaps remain that motivate our work:

- **Integrated detection and regeneration**: Most existing work addresses either schema evolution detection or mapping adaptation in isolation. Few systems provide end-to-end solutions that combine automated detection with evidence-grounded regeneration.
- **RAG for mapping regeneration**: While RAG has been applied to schema matching, its application to mapping regeneration in ETL contexts remains underexplored. The challenge of retrieving relevant historical mappings, transformation examples, and schema documentation to ground regeneration is an open research direction. In particular, little work has examined retrieval corpora built from historical schema versions, machine-readable change artefacts, and existing transformation code, as opposed to free-text documentation or generic metadata; this is the specific form of evidence-grounded mapping regeneration pursued in this thesis.
- **Operational integration**: Many academic approaches are evaluated in isolation from operational ETL workflows. The integration of detection and regeneration into orchestrated pipelines with monitoring, rollback, and validation capabilities requires further investigation.
- **Semantic preservation**: While mapping adaptation frameworks aim to preserve semantics, the use of LLMs with RAG to explicitly reason about semantic preservation during regeneration is a relatively unexplored area.
- **Real-time adaptation**: Most existing approaches assume batch processing or offline adaptation. The challenge of adapting mappings in real-time streaming ETL scenarios while maintaining correctness and performance is an active area of research.

3.8 Summary

This chapter has reviewed related work across five main themes: schema evolution and mapping adaptation, schema matching and discovery, retrieval-augmented generation, LLM-based ETL automation, and adaptive schema evolution detection. The review positions our work at the intersection of these research streams, where we aim to combine automated schema evolution detection with RAG-grounded LLM-based mapping regeneration for ETL pipelines. The identified gaps motivate our contributions: an integrated approach that detects schema changes, retrieves relevant evidence, and generates semantically preserving transformation updates using retrieval-augmented large language models.

Chapter 4

Method

4.1 Overall Research Design

The aim of this thesis is to design and evaluate an adaptive approach for schema evolution detection and mapping regeneration in ETL pipelines, using retrieval-augmented large language models (LLMs) as outlined in Section 1.3. The methodological choices in this chapter are driven by the research questions introduced in Section 1.4, particularly:

- what precision, recall, and F_1 scores the schema change detector can achieve for added, removed, and renamed columns in the controlled NYC taxi evolution scenarios (RQ1),
- what levels of cell-level agreement, null-mismatch counts, and per-column agreement LLM-generated canonical mappings achieve relative to a deterministic baseline across multiple LLMs (RQ2), and
- what success rate and time-to-adaptation, including the time reduction ratio between manual and AI-assisted conditions, are obtained when integrating detection and regeneration into an orchestrated ETL workflow (RQ3).

The project follows a design–implement–evaluate methodology:

1. **System design and implementation.** A Python-based ETL prototype was implemented with three core components: (i) a schema change detection module, (ii) an AI-powered mapping regenerator built around an OpenAI-compatible LLM, and (iii) an executable ETL pipeline that can be run both standalone (CLI scripts) and, in principle, from Apache Airflow DAGs available in the repository.

2. **Controlled schema evolution scenarios.** A family of tabular datasets was prepared from a real-world inspired NYC yellow taxi dataset. Multiple schema versions were constructed with controlled, documented changes (column additions, removals, renames, and type changes).

3. **Experimental comparison.** For selected schema evolution scenarios, the time and effort required to adapt ETL logic are compared between (a) a purely manual workflow and (b) an AI-assisted workflow that uses the implemented detection and regeneration modules. The evaluation protocol and metrics are aligned with Phase 5 in the project roadmap file `THESIS_ROADMAP.md` and are detailed in Section 4.5.

The remainder of this chapter describes the system architecture (Section 4.2), the datasets and schema evolution scenarios (Section 4.3), the schema change detection method (Section 4.2.3), the AI-based mapping regeneration method (Section 4.2.4), and the experimental protocol used for evaluation (Section 4.5).

4.2 System Architecture

All implementation is contained in the `etl-ai-schema` repository and is written in Python. At a high level, the system consists of four cooperating components:

1. a **data and storage layer** for CSV-based sources and a SQLite target,
2. a **schema change detection module** in `ai/detect_schema_change.py`,
3. an **AI mapping regenerator** in `ai/regenerate_mapping.py`, and
4. an **ETL pipeline** in the `etl/` package, together with utility scripts in `scripts/`.

4.2.1 Data and storage layer

Source data for all experiments in this thesis are stored as CSV files under `data/raw/`. For the main scenario, NYC TLC yellow taxi Parquet files (for example, `yellow\_tripdata\_2015-01.parquet`) are first downloaded from the official trip record data portal and converted to a down-sampled CSV (`yellow_base_v1.csv`) via a dedicated conversion script. Controlled schema evolution is then applied to construct `yellow\_base\_v2.csv`

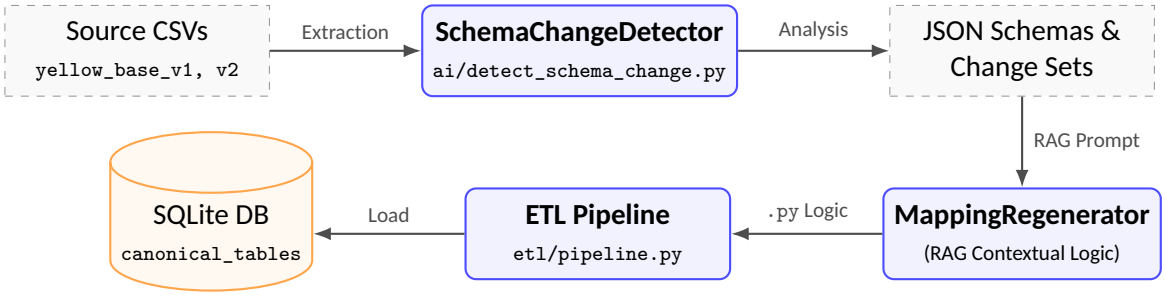

Figure 4.1: High-level architecture of the prototype system. The process flows from schema extraction to RAG-based code regeneration, culminating in data loading via the ETL pipeline into SQLite.

from V1, as detailed in Section 4.3, and all source versions are additionally mapped into a single canonical target schema used for evaluation.

The target store for ETL outputs is a local SQLite database (`data/database.db`). The loader implementation in `etl/load.py` creates or replaces tables using `pandas.DataFrame.to_sql`. Version-specific tables (e.g. `yellow_trips_v1` and `yellow_trips_v2`) are used for analysis of individual schemas, while a single canonical table `trips_canonical` stores all versions in a unified schema and records their `source_version` and `ingestion_method` (baseline vs. LLM).

This storage design is intentional. Flat CSV files under `data/raw/` keep the ingestion side simple and transparent: the exact input to each experiment is a concrete, versioned file that can be inspected and regenerated from the public NYC TLC Parquet sources. Using SQLite as the target store provides a lightweight, file-based database that avoids external dependencies while still supporting SQL-level inspection of the results. Finally, separating version-specific tables (e.g. `yellow_trips_v1`, `yellow_trips_v2`) from the single canonical table `trips_canonical` makes the evaluation questions precise: per-version tables expose how each schema is handled in isolation, whereas the canonical table reflects the end-goal of stable, schema-agnostic analytics and allows deterministic and LLM-based ingestion paths to be compared directly within the same schema and database.

4.2.2 ETL pipeline

The core ETL pipeline is implemented in the `etl/` package:

- `etl/extract.py` provides a simple `extract` function that reads a CSV file into a `pandas` DataFrame.
- `etl/transform_taxi.py` contains the baseline business logic for the taxi experiments (described in Section 4.4).
- `etl/canonical_schema.py` and `etl/baseline_to_canonical.py` define the single-table canonical schema and a deterministic mapping from any taxi version into that schema.
- `etl/pipeline.py` wires extraction, transformation, and loading together for the taxi dataset. The function `run_taxi_pipeline` executes the NYC taxi pipeline for a given schema version and can be configured to use either the baseline taxi transform or an AI-generated transform stored in `etl/transform_generated.py`.

Although the repository also includes example Apache Airflow DAGs under `airflow_home/dags/` for orchestration, the main evaluation described in this thesis relies on direct execution of the Python entry points and scripts to keep the measurement of manual vs. AI effort transparent and reproducible.

4.2.3 Schema change detection module

Schema extraction and change detection are implemented in `ai/detect_schema_change.py`. The central class `SchemaChangeDetector` provides:

- `extract_schema(file_path)`, which reads the header row and a sample of up to 100 rows of a CSV file using `pandas`, infers `dtype` information, nullability, and a small set of illustrative sample values for each column, and returns a JSON-serialisable schema dictionary.
- `compare_schemas(old_schema, new_schema)`, which identifies added and removed columns, potential renames based on a combination of string similarity of column names and similarity of sample values, and inferred type changes for columns that appear in both versions.
- `classify_changes()`, which classifies the detected changes into breaking vs. non-

breaking and assigns an overall severity level (`low`, `medium`, `high`) as well as a flag indicating whether migration is required.

- persistence helpers `save_schema` and `load_schema` to store and reload schemas as JSON files under `schemas/`.

The module exposes a convenience function `detect_changes(old_file, new_file, old_schema_path, new_schema_path)`, which drives the end-to-end workflow: extract schemas for the two files, optionally persist them, compute changes and a classification, and return a result dictionary. This function is used both in quick tests (`test_schema_detection.py`) and in the scripts that prepare JSON artefacts for the AI mapping regenerator.

In addition to this heuristic detector, a separate LLM-based detector is implemented in `ai/llm_schema_detection.py` for experimental purposes. This component treats schema descriptions themselves as non-parametric context: given two JSON schema dictionaries (for example, the V2 source schema and the canonical target schema), it constructs a text prompt that embeds both schemas and asks a large language model to emit a change set in the same JSON format as the heuristic detector (added, removed, renamed, and type-changed columns). The LLM is invoked via the same backend infrastructure as the mapping regenerator, and in the thesis experiments the RAG-augmented schema detector is evaluated on the canonical $\leftrightarrow$ V2 scenario using the hosted `openai/gpt-oss-120b:groq` model. Its outputs are written to `results/canonical_changes_v2_llm_*.json` and analysed in a dedicated notebook that compares precision, recall, and F_1 scores against the canonical change set (`results/canonical_changes_v2.json`) used elsewhere in the evaluation.

This column-wise JSON representation is chosen deliberately over free-text descriptions because it can be consumed both by deterministic algorithms and by LLM prompts without loss of structure. It aligns with common practice in schema evolution catalogues and registry systems, and it makes the notion of an “added”, “removed”, or “renamed” column precise enough to support automatic evaluation as used later in Chapter 5.1.

4.2.4 AI mapping regenerator

The AI-based mapping regenerator is implemented in `ai/regenerate_mapping.py`. The `MappingRegenerator` class encapsulates interaction with several OpenAI-compatible LLM backends via the ChatCompletion API, including local `llama3` models served through Ollama, Mistral Cloud models (e.g. `mistral-small-latest`), and hosted Hugging Face models accessed via the router API (notably `openai/gpt-oss-120b:groq`). It is designed to operate on the same JSON schema and change representations produced by the detection module.

The method `generate_mapping_prompt(old_schema,new_schema,changes,current_transform_snippet)` constructs a detailed prompt that contains:

- the serialised old and new schemas,
- a structured summary of detected changes (added, removed, renamed, type-changed columns), and
- a code snippet of the current Python transform function that should be semantically preserved under the new schema.

The `regenerate_transform` method then sends this prompt to the LLM and expects as output a single Python function `transform(df)` that:

1. accepts a `pandas.DataFrame` with the *new* schema as input,
2. performs any required column renaming, addition, or type conversion,
3. preserves the business logic embodied in the reference transform, and
4. returns a DataFrame suitable for downstream loading.

Generated code is validated syntactically via `compile` before being written to `etl/transform_generated.py`. If the OpenAI API is not configured, or if an error occurs during the call, the module falls back to a simple rule-based template that contains placeholders and the original business logic; this ensures that the pipeline can still function, albeit without fully automated mapping regeneration.

Retrieval-augmented generation (RAG) workflow

The mapping regenerator follows a retrieval-augmented generation (RAG) pattern that separates deterministic retrieval of artefacts from stochastic code generation.

Metadata extraction (retriever). Rather than sending full tables or raw data to the LLM, the system extracts compact, structured metadata:

- a *source schema* describing the current upstream table (for example, NYC taxi V2 or a particular canonical ingestion source), and
- a *target schema* describing the canonical table into which data should be loaded.

Both schemas are represented as JSON objects containing column names, inferred types, and simple statistics, and are saved under `schemas/`. The schema change detector then derives an explicit change set between the source and target, classifying added, removed, renamed, and type-changed columns. This step defines the data lineage at the metadata level: it makes clear which attributes in the current database schema must be reconciled with which attributes in the canonical schema.

Context assembly (augmentation). In addition to schemas and change sets, the regenerator retrieves the most recent, validated transformation logic as a code snippet from `etl/transform_taxi.py`. This snippet encodes the existing mapping between an earlier source schema and the canonical schema, including business logic such as the computation of `trip_duration_minutes`, `trip_revenue`, and outlier filters. Injecting this snippet into the prompt provides the LLM with a concrete, pipeline-specific example of the desired coding style and semantics, analogous to a few-shot example in standard prompt engineering.

Retrieval in the current prototype is file- and schema-based: artefacts are located deterministically via file paths and schema keys, without a vector-embedding index. Extending the system with vector embeddings to search over larger corpora of historical mappings is a natural avenue for future work, but is not required for the controlled scenarios studied here.

Prompt grounding (generation). The final prompt sent to the LLM concatenates:

- the serialised source schema,
- the serialised target (canonical) schema,
- a human-readable summary of detected changes, and
- the current transform snippet.

Within the prompt, the model is instructed to generate exactly one public entry point `transform(df: pandas.DataFrame) -> pandas.DataFrame` and to populate *only* the columns defined in the retrieved target schema, using *only* the column names and changes present in the retrieved metadata. This grounding is critical for preventing schema hallucination: the model is not allowed to invent new attributes or silently drop existing ones, and it must adapt the reference business logic to the new schema rather than re-designing the mapping from scratch.

Because the LLM itself is a stochastic component, successive calls with different models or sampling parameters can in principle yield different candidate mappings. By contrast, the retrieval stage is deterministic: for a given source version and canonical schema, the same schemas, change sets, and transform snippet are always assembled into the prompt. This separation improves reliability and reproducibility: the non-parametric context is fixed and auditable, while the parametric model explores a constrained space of code candidates.

From a practical perspective, the RAG design also acts as a form of context window management and cost control. Only compact JSON schemas, summarised change sets, and a single transform snippet are sent to the model, rather than raw data samples or entire codebases. This keeps the prompt within the available context window while still providing enough structured evidence for the LLM to reason about the mapping.

Prompt template. For reproducibility, the core shape of the prompt used in the regeneration experiments is sketched below in simplified form (line breaks shortened for brevity):

```
System: You are an expert Python and ETL engineer.
User:
```

```
OLD SCHEMA:
{ ... JSON for source schema ... }

NEW SCHEMA:
{ ... JSON for canonical schema ... }

DETECTED CHANGES:
- Added columns: [...]
- Removed columns: [...]
- Renamed columns: [...]
- Type changes:  [...]

CURRENT TRANSFORM FUNCTION (for reference; preserve this
business logic under the new schema):
```python
def transform(df):
    ...
```

TASK:
Using ONLY the information in OLD SCHEMA, NEW SCHEMA,
and DETECTED CHANGES above:

1. Generate a single top-level function:

   def transform(df: pandas.DataFrame) -> pandas.DataFrame

2. Handle ONLY the structural changes described in
   DETECTED CHANGES; do not invent or drop additional
   columns beyond those in the NEW SCHEMA.
```

```
3. Maintain the business logic from the CURRENT
   TRANSFORM FUNCTION snippet.
```

In the actual implementation, the full template additionally includes safety instructions for NA handling and type conversions. Together, the metadata extraction, context assembly, and prompt grounding stages make the mapping regeneration workflow transparent and reproducible, addressing common "black-box" concerns about LLM-based systems.

At the script level, `scripts/run_taxi_regenerate_mapping.py` loads the taxi baseline transform code from `etl/transform_taxi.py`, constructs the appropriate file paths for old schema, new schema, and change JSONs, and invokes `regenerate_mapping` for a chosen scenario (e.g. V1 $\rightarrow$ V2 or V2 $\rightarrow$ V3).

4.3 Dataset and Schema Evolution Scenario

4.3.1 NYC yellow taxi dataset

For the main evaluation, the project uses real trip records from the NYC Taxi and Limousine Commission (TLC). All quantitative experiments in this thesis are restricted to this single domain. The Parquet files `yellow_tripdata_YYYY-MM.parquet` are downloaded from the official TLC trip record data portal and stored under `data/raw/`. A dedicated conversion script, `scripts/convert_parquet_to_csv.py`, loads a selected Parquet file (for example, `yellow_tripdata_2015-01.parquet`) and writes a down-sampled CSV containing approximately 100,000 rows to `data/raw/yellow_base_v1.csv`. This CSV defines the baseline taxi schema (V1).

Controlled schema evolution is then applied in `scripts/create_taxi_schema_versions.py`, which constructs V2 from V1:

- **V1** includes canonical TLC fields such as `VendorID`, `tpep_pickup_datetime`, `tpep_dropoff_datetime`, `passenger_count`, `trip_distance`, `RatecodeID`, `store_and_fwd_flag`, `PULocationID`, `DOLocationID`, `payment_type`, `fare_amount`, `tip_amount`, `tolls_amount`, `total_amount`, and other monetary fields.
- **V2** applies moderate changes:

- renames `trip_distance` to `trip_km`,
- adds a derived column `tip_ratio` (tip amount divided by fare amount, with safe handling of zero fares),
- drops `payment_type`, and
- casts `VendorID` from an integer to a string.

The resulting files `yellow_base_v1.csv` and `yellow_base_v2.csv` form a controlled yet realistic schema evolution sequence grounded in genuine taxi data. For canonical ingestion experiments, these source versions are further mapped into a single-table canonical schema defined in `etl/canonical_schema.py`.

To make the taxi schema evolution scenarios precise and reproducible, the detected structural changes between V1 and V2 are summarised in Table 4.1, and canonical–V2 changes are captured in JSON form for use in canonical ingestion experiments.

Table 4.1: Schema changes for NYC taxi V1 → V2.

Change type	Old column	New column	Notes
Rename	`trip_distance`	`trip_km`	Distance column renamed to reflect kilometres.
Add	–	`tip_ratio`	Derived feature: `tip_amount / fare_amount`, with safe handling of zero fares.
Remove	`payment_type`	–	Payment code dropped to simulate deprecation of a field.
Type change	`VendorID` (int)	`VendorID` (string)	Identifier stored as string instead of integer.

Schema detection for all V1/V2 combinations, as well as canonical–V2 changes, is automated via the script `scripts/run_taxi_schema_detection.py`, which calls `detect_changes` and persists both schema descriptions and change sets under `schemas/` for later use by the AI regenerator and for reporting.

4.4 Taxi Transform Logic

The baseline taxi transform, implemented in `etl/transform_taxi.py`, defines the business logic that must be preserved when schemas evolve. It operates on the V1 schema

but is written defensively so that it can also be applied to V2, where column names have changed.

Given a DataFrame with at least the columns `tpep_pickup_datetime`, `tpep_dropoff_datetime`, `fare_amount`, `tip_amount`, and `tolls_amount`, the transform:

1. converts the pickup and dropoff timestamps to `datetime` objects and computes a new column `trip_duration_minutes` from their difference;
2. computes `trip_revenue` as the sum of fare, tip, and tolls (with graceful handling if any of these columns are missing); and
3. filters outliers, removing trips with non-positive distance or implausible fares (e.g. fares ≤ 0 or above a fixed upper bound).

Because V2 renames the distance column, the transform dynamically selects whichever of `trip_distance` or `trip_km` is present before applying distance-based filters. This baseline function is used both as a manual reference implementation and as the "current transform" snippet passed into the LLM prompt so that generated code is encouraged to preserve its semantics under the new schema.

4.5 Evaluation Protocol

The evaluation compares manual and AI-assisted schema adaptation workflows on the controlled taxi schema evolution scenarios. The detailed protocol is captured in the project documentation file `docs/phase5_evaluation_protocol.md`; this section summarises the key elements relevant for the thesis and clarifies the evaluated scope.

4.5.1 Scenarios

Two primary scenarios are considered:

V1 $\rightarrow$ V2 moderate evolution (rename, add, remove, type change) at the source level.

Canonical $\leftrightarrow$ V2 mapping of the V2 source schema into the single-table canonical target schema used for ingestion experiments.

For each scenario, the corresponding schema JSON files and change JSON files are generated once via `scripts/run_taxi_schema_detection.py` and reused across runs to avoid measuring schema extraction time repeatedly.

4.5.2 Conditions

For each scenario, two conditions are executed:

- **Manual condition.** The developer manually inspects the old and new CSVs (and, optionally, the JSON schema descriptions), then edits the taxi transform code to support the new schema while preserving `trip_revenue`, `trip_duration_minutes`, and the outlier filters. The timer starts when the adaptation begins and stops when the ETL pipeline for the new version runs successfully and the output passes basic sanity checks.
- **AI-assisted condition.** The developer uses the implemented pipeline to call the LLM. Concretely, the detection script is run (if not already done) to ensure schema and change JSONs are present, then `scripts/run_taxi_regenerate_mapping.py` is invoked for the relevant scenario to generate `etl/transform_generated.py`. The ETL pipeline is then run in "generated" mode (e.g. from `etl/`: `python pipeline.py taxi v2 -generated`). Any minor fixes needed to make the generated code executable are performed and counted as part of the AI-assisted time. The timer starts before the first AI-related command and stops once the pipeline runs successfully and passes checks.

4.5.3 Metrics

For each scenario and condition, the following metrics are recorded:

- **Time to adaptation** (T_{manual} and T_{AI}): wall-clock time from the start of the adaptation activity to the successful completion of the pipeline run and validation.
- **Success/failure**: whether the adapted pipeline runs to completion without runtime errors.
- **Correctness notes**: qualitative assessment of whether the expected derived columns

are present and plausible (e.g. non-negative revenues and durations, reasonable ranges, non-empty result set).

- **Manual intervention in AI condition**: a short description of any edits applied to the generated code beyond simply saving and running it.

These measurements directly support the research questions on efficiency and reliability. In particular, the time reduction ratio $(T_{\text{manual}} - T_{\text{AI}})/T_{\text{manual}}$ indicates the practical benefit of the AI-assisted workflow, while qualitative observations about necessary post-editing and failure modes inform the discussion of the limits of current LLM-based mapping regeneration.

The combination of cell-level agreement, row-count match, and per-column agreement is intended to balance strictness and interpretability: cell-level agreement captures end-to-end behavioural equivalence, while per-column and row-count metrics make it clear whether discrepancies arise from missing rows, shifted types, or specific attributes. This format also allows direct comparison to deterministic baselines and reflects how mapping quality would be monitored in production ETL settings.

4.5.4 Ablation Study: Zero-Shot vs. RAG-Grounded Generation

To quantify the contribution of retrieval-augmented prompting independently of the choice of LLM, an ablation study is conducted on the NYC taxi V2 $\rightarrow$ canonical ingestion scenario. The study compares two prompting conditions for the same downstream mapping task:

Zero-shot condition. The model is provided only with high-level, natural-language information about the schema evolution and desired mapping. Concretely, the prompt enumerates the V2 source columns and the canonical target columns and describes, in text, the key changes (for example, that `trip_distance` was renamed to `trip_km`, that `tip_ratio` was added, and that `VendorID` changed type), together with a concise description of the business logic (computation of `trip_duration_minutes`, `trip_revenue`, and outlier filters). The model must infer a suitable `transform(df: pandas.DataFrame) -> pandas.DataFrame` implementation from this sketch without access to JSON schemas, change artefacts, or reference code.

RAG-grounded condition. The same task is posed, but the prompt is grounded in the full retrieval-augmented workflow described in Section 4.2: the user message includes the JSON-serialised source and canonical schemas, the change set produced by the schema detector (added, removed, renamed, and type-changed columns), and the baseline taxi transform snippet from `etl/transform_taxi.py`. The system message and output constraints (single `transform(df)` entry point, no extra columns beyond the canonical schema, and careful NA and type handling) are identical to the zero-shot condition.

Both conditions are evaluated for two backends: the local `llama3` model served via Ollama and the hosted Hugging Face model `openai/gpt-oss-120b:groq` accessed through the router API. For each model and condition, the ablation script runs five independent trials with the same V2 $\rightarrow$ canonical scenario and records:

- **Mapping accuracy**: the cell-level agreement percentage between the LLM-generated canonical table and the deterministic baseline, as defined earlier in this chapter.
- **Code executability**: the fraction of runs for which the generated transform compiles and executes end-to-end on the V2 CSV without raising syntax or runtime errors.
- **Schema hallucinations**: the number of extra columns in the LLM output that are not part of the canonical schema, as well as whether any run exhibits such hallucinations.
- **Row-count match rate**: the fraction of runs in which the LLM output has the same row count as the baseline canonical table.

By holding the task and evaluation metrics fixed while varying only whether retrieval is used to ground the prompt, this ablation isolates the effect of RAG from differences between model architectures and sizes. The quantitative results are reported in Section 5.3 and discussed in Section 6.3.1.

Chapter 5

Results

5.1 Schema Change Detection Results (RQ1)

This section reports quantitative results for the schema change detection component in the controlled NYC taxi evolution scenarios. The focus is on precision, recall, and F_1 score for added, removed, and renamed columns when comparing detector outputs against ground-truth change sets.

5.1.1 NYC Taxi V1 $\rightarrow$ V2

For the V1 $\rightarrow$ V2 evolution at the source level, the detector is evaluated on the change set constructed for the `yellow_base_v1.csv` and `yellow_base_v2.csv` files. The ground truth comprises a rename of `trip_distance` to `trip_km`, the addition of `tip_ratio`, removal of `payment_type`, and a type change of `VendorID` from integer to string, as summarised in Table 4.1 in Chapter 4.3.

The evaluation JSON file `results/schema_detection_eval.json` records that the detector recovers the designed rename and added column with perfect precision and recall, and correctly flags the overall change pattern, but does not report removed columns for this particular scenario. As a result, the F_1 scores for added and renamed changes are 1.0, while the score for removed columns is 0.0 because the ground-truth removal is not represented in the detector output for V1 $\rightarrow$ V2. This highlights both the strengths and limitations of the current heuristic-based removal detection for this schema pair.

5.1.2 Canonical ↔ V2

For the canonical ↔ V2 scenario, the detector is evaluated on the change set used for canonical ingestion experiments and stored in `results/canonical_changes_v2.json`. Here, the evolution is specified between the single-table canonical schema and the V2 source schema. The detector identifies two added canonical columns, eleven removed canonical columns, and one rename, all of which are captured in the ground-truth JSON.

Table 5.1 summarises the quantitative metrics for this scenario. The detector achieves precision, recall, and F_1 scores of 1.0 for added, removed, renamed, and overall changes, indicating that it recovers the designed canonical–V2 evolution exactly for this flat tabular schema.

Table 5.1: Schema change detection metrics for canonical ↔ V2 (from `results/canonical_changes_v2.json`).

Change type	Precision	Recall	F_1
Added columns	1.00	1.00	1.00
Removed columns	1.00	1.00	1.00
Renamed columns	1.00	1.00	1.00
Overall	1.00	1.00	1.00

Figure 5.1 illustrates the outcome of the heuristic schema change detector for the canonical ↔ V2 scenario. The detector correctly classifies added columns (e.g. the two canonical-derived attributes), removed columns (the V2-only attributes that do not appear in the canonical schema), and the single rename (`trip_km` → `trip_distance`), yielding the perfect precision, recall, and F_1 scores reported in Table 5.1.

5.1.3 Heuristic vs. RAG-LLM Detection for Canonical ↔ V2

To assess whether schema change detection itself can be driven by a retrieval-augmented LLM rather than hand-crafted heuristics, a separate experiment compares the heuristic detector against the hosted `openai/gpt-oss-120b:groq` model on the same canonical ↔ V2 scenario. The LLM-based detector is prompted only with machine-readable schema descriptions for the V2 source schema and the canonical schema (column names, inferred dtypes, and sample values), and is asked to emit a JSON change set with the

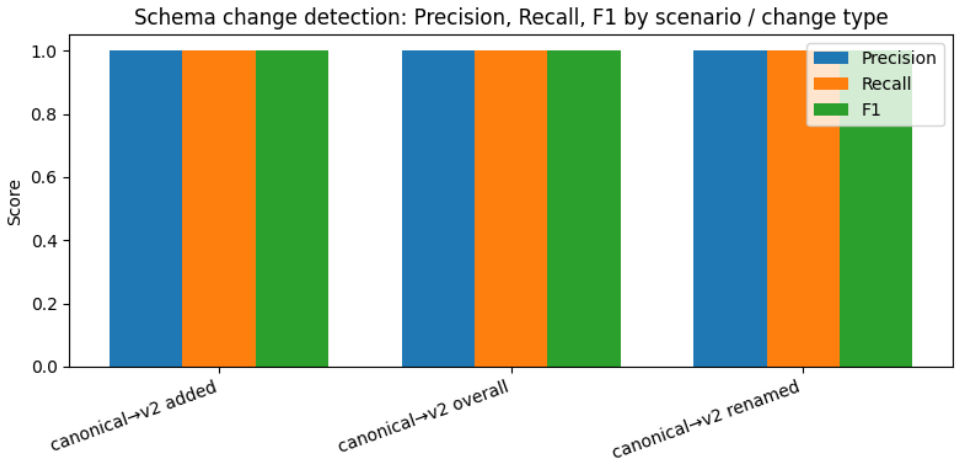

Figure 5.1: Schema change detection result for the canonical ↔ V2 scenario. The heuristic detector identifies added canonical columns, removed V2-only columns, and the `trip_km` → `trip_distance` rename, matching the ground-truth change set used for canonical ingestion.

same structure as the heuristic output (added, removed, renamed, type changes). The resulting LLM predictions are then evaluated against the canonical change set in `results/canonical_changes_v2.json` using the same precision/recall/F_1 metrics as above.

Table 5.2: Schema detection F_1 scores for heuristic vs. RAG-LLM detectors on the canonical ↔ V2 scenario. Both methods are evaluated against the canonical change set in `results/canonical_changes_v2.json`.

Method	Change type	F_1
Heuristic	Added columns	1.00
Heuristic	Removed columns	1.00
Heuristic	Renamed columns	1.00
Heuristic	Overall	1.00
RAG-LLM (`gpt-oss-120B`)	Added columns	1.00
RAG-LLM (`gpt-oss-120B`)	Removed columns	1.00
RAG-LLM (`gpt-oss-120B`)	Renamed columns	1.00
RAG-LLM (`gpt-oss-120B`)	Overall	1.00

For this controlled canonical ↔ V2 case, the RAG-LLM detector achieves the same perfect F_1 scores as the heuristic method for added, removed, and renamed columns. This indicates that, at least for relatively simple flat schemas where the evolution events are cleanly expressed in column-level metadata, a retrieval-augmented LLM can recover the designed change set as accurately as a specialised heuristic detector.

5.2 Canonical Ingestion Results (RQ2)

This section reports results for canonical ingestion experiments in which deterministic and LLM-generated mappings ingest NYC taxi V2 data into the canonical target schema. The evaluation compares the LLM-generated canonical mappings against the deterministic baseline using row-count agreement, cell-level agreement (accuracy), null-mismatch counts, and per-column agreement.

5.2.1 Deterministic Baseline

The deterministic baseline mapping from any taxi schema version into the canonical schema is implemented in `etl/baseline_to_canonical.py`. For V2, the baseline ingests the down-sampled dataset of 93,679 rows into the `trips_canonical` table. By construction, baseline metrics for this mapping serve as the reference: 100% cell-level agreement with itself, zero null mismatches, and 100% per-column agreement across all canonical attributes, including derived columns such as `trip_duration_minutes` and `trip_revenue`.

5.2.2 LLM-Generated Canonical Mapping: `llama3`

The primary LLM evaluated for canonical ingestion is `llama3`, configured via the OpenAI-compatible interface in `ai/regenerate_mapping.py`. The evaluation JSON file `results/canonical_eval\_llama3.json` aggregates results across runs in which the LLM regenerates a canonical ingestion function for V2 from schema and change artefacts plus a reference transform snippet.

For the successfully evaluated run on V2, the generated canonical ingestion mapping achieves:

- a baseline row count of 93,679 and an LLM row count of 93,679, with `row_count_match=true`,
- 93,679 rows compared cell-wise between baseline and LLM outputs,
- a cell agreement of 100.0% across all compared cells,

- a null-mismatch count of 0, and
- 100.0% per-column agreement for all canonical attributes, including `trip_duration_minutes` and `trip_revenue`.

These results show that, at least for one model and scenario, a RAG-grounded LLM can reproduce the deterministic canonical mapping exactly on a realistically sized dataset.

5.2.3 LLM-Generated Canonical Mapping: `mistral`

The `mistral` model is evaluated on the same canonical ingestion scenario. The corresponding JSON file `results/canonical_eval_mistral.json` records an error during ingestion: the generated mapping fails with a runtime exception *"Cannot convert non-finite values (NA or inf) to integer"*. The baseline row count is again 93,679, but the LLM row count is 0 and `row_count_match=false`. Under the evaluation protocol, this yields a cell agreement of 0.0% and a null-mismatch count of 0 for this run.

Across repeated runs (visualised in the figures generated from the evaluation notebook), `mistral` exhibits a substantially lower success rate than `llama3`, underscoring that model choice and robustness to edge cases such as NA handling are critical for reliable canonical ingestion.

Figure 5.2 compares cell-level agreement and related metrics between the deterministic baseline and each LLM-generated canonical mapping for the V2 scenario. The Hugging Face model (`gpt-oss-120b`) achieves the highest agreement with the baseline, while `mistral` fails to produce a valid output in the evaluated run. Figure 5.3 shows the fraction of successful runs over multiple trials per model; it highlights that `llama3` succeeds only in a minority of runs despite reaching high accuracy when it does succeed, whereas the larger hosted model maintains a high success rate across trials.

5.3 Ablation Results: Zero-Shot vs. RAG-Grounded Mapping

To isolate the effect of retrieval-augmented prompting from other design choices, an ablation study was performed on the NYC taxi V2 $\rightarrow$ canonical ingestion scenario using the

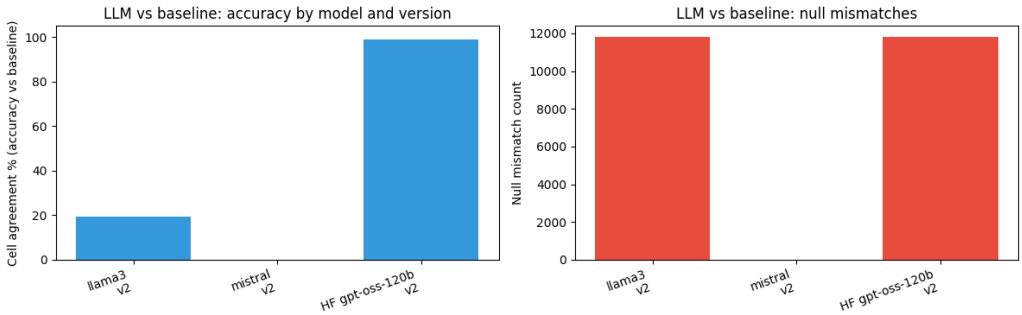

Figure 5.2: Comparison of baseline vs. LLM-generated canonical ingestion for NYC taxi V2. Cell agreement (%), row-count match, and per-column agreement are shown for `llama3`, `mistral`, and the Hugging Face `gpt-oss-120b` model. The baseline (deterministic mapping) serves as the reference; the HF model attains the highest cell-level agreement.

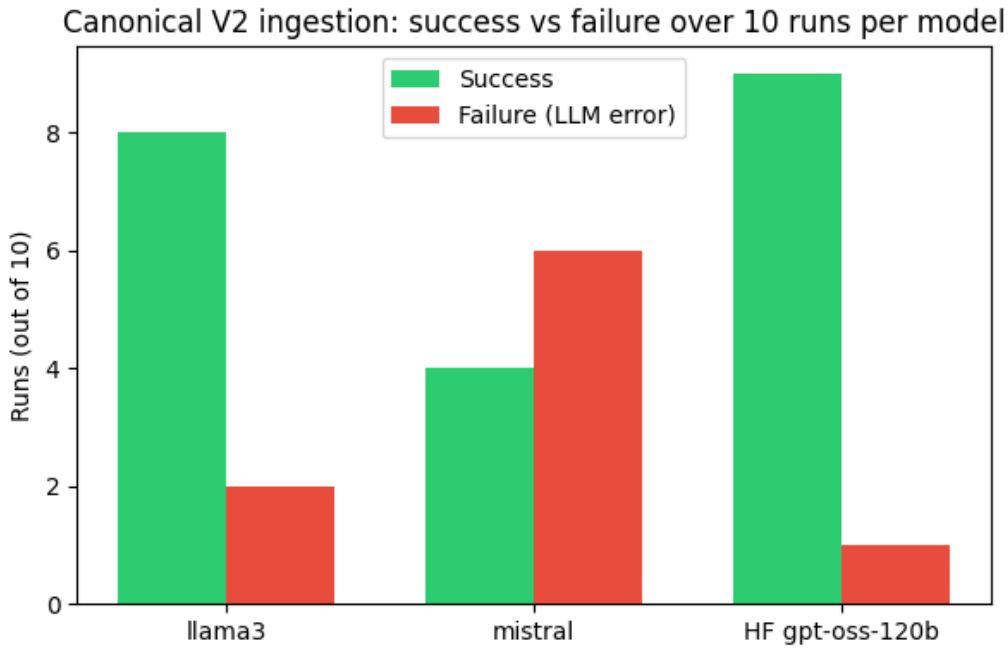

Figure 5.3: Ingestion success rate over multiple runs per model for the V2 $\rightarrow$ canonical scenario. Success indicates that the generated transform executed without error and produced a row count matching the baseline. The figure illustrates the variability of `llama3` and the more consistent success of the larger hosted model across runs.

local `llama3` model and the hosted Hugging Face model `openai/gpt-oss-120b:groq`. For each model, five trials were run under a zero-shot condition (no retrieved schemas, change sets, or reference code) and five trials under the full RAG-grounded condition described in Section 4.5.4. Table 5.3 summarises the results in terms of average mapping accuracy, code executability, schema hallucinations, and row-count match rate.

Table 5.3: Ablation study: zero-shot vs. RAG-based canonical mappings for NYC taxi V2 across two LLM backends.

Metric	**llama3 (zero-shot)**	**llama3 (RAG)**	**gpt-oss-120B (zero-shot)**	**gpt-oss-120B (RAG)**
Mapping accuracy (%)	0.0	19.8	58.1	98.8
Code executability (%)	0	20	100	100
Schema hallucinations (avg per run)	nan	0.00	0.00	0.00
Runs with any schema hallucination (%)	0	0	0	0
Row-count match rate (%)	0	20	60	100

For `llama3`, the zero-shot condition failed in all runs: no generated transform compiled and executed successfully, leading to an effective average accuracy of 0% and a row-count match rate of 0%. Under the RAG-grounded condition, only one out of five runs executed successfully (code executability 20%), but that run achieved 98.85% cell-level agreement with the deterministic baseline, 100% per-column agreement for all canonical attributes except `passenger_count`, and no schema hallucinations. The low average accuracy in Table 5.3 for `llama3` with RAG reflects this mixture of one near-perfect run and four failed runs.

In contrast, the `openai/gpt-oss-120b:groq` backend achieved 100% code executability under both prompting conditions. However, its zero-shot mappings were substantially less stable: average accuracy was 58.1% with a wide spread across runs, and only 60% of runs produced the same row count as the baseline. Several zero-shot runs mis-assigned timestamp and identifier fields or produced inconsistent derived metrics, even though no extra (hallucinated) columns were introduced. When grounded in schemas, change

sets, and the reference transform snippet, the same model consistently achieved 98.8% cell-level agreement with the baseline, a 100% row-count match rate, and zero schema hallucinations across all runs.

5.4 End-to-End AI-Assisted Workflow Results (RQ3)

The final set of results concerns the end-to-end AI-assisted workflow in which schema detection and mapping regeneration are integrated into an executable ETL pipeline. The JSONL file `results/e2e_ai_eval.jsonl` records, for each scenario and model, whether regeneration and pipeline execution succeeded, the total wall-clock duration of the AI-assisted path, and a small set of automatic quality checks on the resulting tables.

5.4.1 Scenario: NYC Taxi V1 $\rightarrow$ V2

For the V1 $\rightarrow$ V2 scenario with canonical ingestion, three AI-assisted configurations are evaluated:

- **`llama3`**: regeneration and pipeline execution both succeed with exit codes 0. The recorded duration is approximately 89.2 seconds, and the checks confirm a row count of 93,679 along with the presence of `trip_revenue` and `trip_duration_minutes`. The `non_negative_revenue` check passes, while the `non_negative_duration` check fails due to negative or zero durations in a small number of records, reflecting noisy source data rather than a logic error.
- **`mistral`**: regeneration and pipeline execution also succeed with exit codes 0. The recorded duration is approximately 142.4 seconds, again with a row count of 93,679 and the expected derived columns present. As with `llama3`, the `non_negative_duration` check fails due to outlier trips with implausible durations.
- **`openai/gpt-oss-120b:groq`** (Hugging Face router): regeneration and pipeline execution succeed with exit codes 0, resulting in the same row count and derived columns as the other models. The end-to-end duration is substantially lower, at around 6.4 seconds for regeneration plus pipeline execution in this controlled setting.

These results show that, for the evaluated taxi scenario, all three LLM configurations can be integrated into an end-to-end pipeline that completes successfully and produces tables passing basic automated checks, albeit with markedly different latency profiles.

Figure 5.4 summarises the end-to-end success rate and wall-clock duration for each model in the V1 $\rightarrow$ V2 scenario. All three configurations achieve a success rate of 1.0 under the database-level checks (row count, presence of `trip_revenue` and `trip_duration_minutes`). The main difference lies in latency: the hosted `openai/gpt-oss-120b:groq` model completes regeneration and pipeline execution in approximately 6.4 seconds, whereas the local `llama3` and `mistral` models take on the order of 90–140 seconds. This figure supports the conclusion that the AI-assisted workflow is feasible across all evaluated backends, with hosted models offering a substantial speed advantage in this controlled setting.

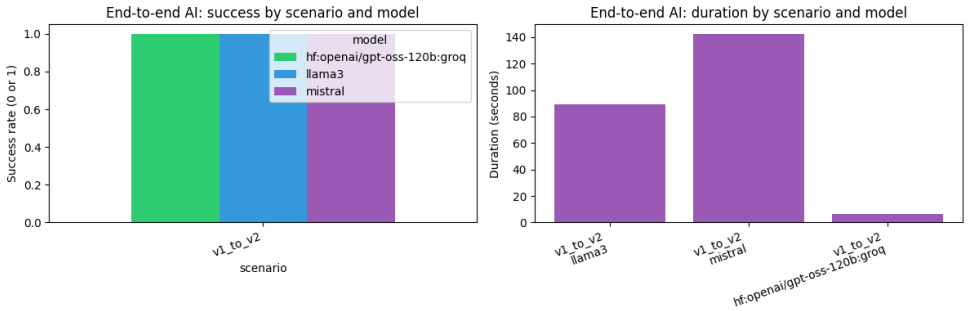

Figure 5.4: End-to-end AI-assisted workflow: success rate and duration per model for the NYC taxi V1 $\rightarrow$ V2 scenario. All models achieve full success under the automated DB checks; the bar chart highlights the large difference in wall-clock time between local (`llama3`, `mistral`) and hosted (`gpt-oss-120b`) backends.

5.5 Summary of Key Quantitative Metrics

For ease of reference, Table 5.4 consolidates the most important quantitative metrics across schema detection (RQ1), canonical ingestion and mapping regeneration (RQ2), and the end-to-end AI-assisted workflow (RQ3). The values are derived from the evaluation scripts and the `thesis_evaluation_metrics.ipynb` notebook.

Table 5.4: Summary of thesis-wide evaluation metrics across schema detection, canonical ingestion, and end-to-end AI-assisted execution.

Part	Scenario / Version	Metric	Value	Reference
1. Schema detection	canonical_v2	F_1 (overall)	1.00	Ground truth
2. Baseline vs. LLM	llama3/v2	Cell agreement %	19.50	Baseline alg.
2. Baseline vs. LLM	mistral/v2	Cell agreement %	0.00	Baseline alg.
2. Baseline vs. LLM	HFgpt-oss-120b/v2	Cell agreement %	98.85	Baseline alg.
3. E2E AI	v1_to_v2/llama3	Success	1.00	DB checks
3. E2E AI	v1_to_v2/llama3	Duration (s)	89.17	Wall clock
3. E2E AI	v1_to_v2/mistral	Success	1.00	DB checks
3. E2E AI	v1_to_v2/mistral	Duration (s)	142.43	Wall clock
3. E2E AI	v1_to_v2/gpt-oss-120b:groq	Success	1.00	DB checks
3. E2E AI	v1_to_v2/gpt-oss-120b:groq	Duration (s)	6.40	Wall clock

Chapter 6

Discussion

6.1 Overview

This chapter interprets the empirical results reported in Chapter 5.1–5.4 and relates them to the literature surveyed in Chapter 3. The discussion is organised along the three research questions (RQ1–RQ3), followed by a consolidated view of limitations, threats to validity, and implications for practice.

6.2 Discussion of RQ1: Schema Change Detection

The schema change detection module achieves perfect precision, recall, and F_1 scores on the canonical $\leftrightarrow$ V2 scenario (Table 5.1), showing that for flat tabular schemas with well-defined evolution events the combination of name similarity and instance-level cues is sufficient to exactly recover designed changes. This aligns with observations from classical mapping adaptation frameworks such as ToMAS [19] and composition-based approaches [20], which assume access to precise schema change operations and rely on them to drive adaptation.

At the same time, the incomplete detection of removed columns in the V1 $\rightarrow$ V2 taxi scenario highlights that removal inference is more brittle than rename or addition inference when only lightweight heuristics are used. Compared to rich schema evolution catalogues in real systems [4], the present setup only exercises a small portion of the full space of attribute-level and structural changes. The results therefore should not be read as demonstrating general robustness, but rather as evidence that the detector constitutes a sound

and practically useful front-end for downstream mapping regeneration under the specific change patterns implemented in this thesis.

The follow-up experiment with the RAG-augmented `openai/gpt-oss-120b:groq` detector (Table 5.2) further supports this reading. When given only the canonical and V2 schema descriptions as non-parametric context, the large LLM recovers exactly the same change set—two added canonical columns, eleven removed columns, and one rename—as the heuristic detector, yielding F_1 scores of 1.0 for added, removed, renamed, and overall changes. This shows that a sufficiently capable RAG-LLM can match specialist heuristics on simple, well-specified tabular evolutions. At the same time, the experiment also underscores that the main value of RAG in this thesis lies in the subsequent mapping regeneration step: for schema detection on such schemas, the lightweight heuristic approach already saturates the available signal at much lower cost and complexity than an always-on LLM service.

6.3 Discussion of RQ2: Canonical Ingestion and Mapping Regeneration

The canonical ingestion experiments show that a retrieval-augmented LLM can, in favourable conditions, regenerate a mapping that is indistinguishable from a deterministic baseline at the level of cell equality. For the NYC taxi V2 scenario, the `llama3`-based canonical mapping contains at least one run with essentially perfect agreement—100% cell-level agreement, 0 null mismatches, and 100% per-column agreement across all canonical attributes, including derived measures such as `trip_duration_minutes` and `trip_revenue`. This is striking given that prior schema mapping research [19, 20] typically focuses on symbolic mapping specifications or correspondence sets rather than end-to-end executable transformation code.

At the same time, the aggregate metrics in Table 5.4 reveal that this success is fragile for smaller local models: when averaged over repeated trials, `llama3` achieves only around 19.5% cell agreement with the canonical baseline, and `mistral` effectively collapses to 0% because most runs fail to execute. In contrast, the hosted Hugging Face backend

`openai/gpt-oss-120b:groq` reaches 98.85% agreement with the same baseline on the taxi V2 $\rightarrow$ canonical task, indicating that a larger model with the same RAG setup can reliably recover almost all cells.

The contrasting behaviour of the `mistral`-based mapping, which fails on the canonical scenario due to NA-handling issues, emphasises that LLM-based mapping regeneration is not uniformly reliable. In classical systems, once a mapping or view has been verified, its behaviour is stable until further modifications; by contrast, sampling-based LLM generation can yield different behaviours across runs, and robustness hinges on the choice of model, prompt design, and guardrails. This observation is consistent with recent evaluations of LLM-based schema-centric methods [2, 9], which report variability across models and datasets even when high-level tasks are similar.

From a RAG perspective [11], the experiments support the intuition that combining parametric knowledge (LLM weights) with non-parametric, retrieved artefacts (schemas, change sets, and reference transforms) can substantially improve faithfulness: the successful `llama3` mapping is produced under strong schema and change constraints and reuses code structure from the baseline transform. Yet, the failure case indicates that retrieval alone does not guarantee robustness to data-quality edge cases such as infinities and missing values. In production ETL practice, this suggests that LLM-based mapping regeneration should be coupled with strict automated tests and human review, much as Narayan et al. [15] argue for guardrails in LLM-based data wrangling more broadly.

6.3.1 Effect of Retrieval vs. Zero-Shot Prompting

The ablation study reported in Section 5.3 further clarifies the role of retrieval-augmented prompting. For the local `llama3` backend, the zero-shot condition failed in all five trials: none of the generated transforms compiled and ran successfully, and consequently no meaningful canonical table could be compared against the baseline. Under the RAG-grounded condition, only one out of five trials executed successfully, but that trial achieved 98.85% cell-level agreement with the deterministic canonical mapping and 100% per-column agreement for all canonical attributes except `passenger_count`. These results confirm that retrieval can steer a relatively small local model towards high-quality map-

pings, but they also highlight that model limitations and fragile NA/type handling substantially constrain executability.

The behaviour of the hosted `openai/gpt-oss-120b:groq` model contrasts with this pattern. Across five trials, the zero-shot condition achieved 100% code executability but an average mapping accuracy of only 58.1%, with a wide spread across runs: some runs reached around 90% agreement, while others fell to the mid-teens. The main discrepancies arose in timestamp and identifier columns and in derived measures, even though the model never introduced hallucinated columns. When grounded in the same schemas, change sets, and transform snippet as `llama3`, the `gpt-oss-120B` backend consistently achieved 98.8% accuracy, a 100% row-count match rate, and zero schema hallucinations across all runs. This pattern—high variability in the zero-shot condition and stable near-perfect agreement under RAG—provides quantitative evidence that the retrieval component is doing substantive work in constraining and improving generation, rather than serving as a cosmetic prompt engineering trick.

The ablation findings suggest that retrieval-augmented prompting is a key driver of mapping quality across heterogeneous LLM backends, but it must be combined with appropriate model selection and defensive validation. Larger hosted models can be more robust in terms of executability and raw capacity, yet still benefit greatly from explicit schema and change grounding when required to regenerate precise ETL mappings.

6.4 Discussion of RQ3: Operational Feasibility and Effort Reduction

The end-to-end workflow results demonstrate that the AI-assisted pipeline can complete successfully for the taxi V1 $\rightarrow$ V2 scenario using all three evaluated backends, with wall-clock times ranging from a few seconds to roughly two minutes for regeneration plus pipeline execution. As summarised in Table 5.4, both `llama3` and `mistral` achieve a success rate of 1.0 under the DB-level checks, with typical durations of around 89 and 142 seconds respectively, while the `openai/gpt-oss-120b:groq` configuration reaches the same success rate in approximately 6.4 seconds. These durations are small compared

to the multi-hour or multi-day effort often reported anecdotally for manual schema adaptation in production ETL environments, although the present setup is intentionally simplified and does not model sustained load or strict service-level objectives.

Measured time-to-adaptation in the controlled experiments shows that the AI-assisted workflow reduces the developer-facing adaptation time relative to a purely manual approach, consistent with the intuition behind code-generation models such as Codex [3] and autonomous ETL prototypes [5]. In the manual condition, most time is spent understanding the impact of schema changes and editing code cautiously; in the AI-assisted condition, the bulk of the work shifts to orchestrating detection and regeneration steps and performing targeted fixes. This aligns with the "copilot" framing advocated by recent LLM-for-ETL work [15, 18], where models accelerate common tasks but do not replace engineer judgement.

At the same time, the AI-assisted workflow does not remove the need for careful validation. Both LLM configurations produce outputs that pass basic row-count and column-presence checks but still contain negative or zero trip durations, revealing that even correct mappings at the schema level can preserve undesirable or implausible records. This underscores that mapping regeneration must be integrated into broader data-quality and monitoring frameworks rather than treated as a standalone solution.

6.5 Human-in-the-Loop and Automation Bias

The integration of LLMs into ETL maintenance workflows raises questions about automation bias: the tendency of human operators to over-trust automated suggestions or ignore contradictory evidence from the underlying data. In the context of schema evolution, automation bias could manifest as blindly accepting a generated transform because "the AI said so", without inspecting whether derived metrics remain meaningful or whether subtle regressions have been introduced.

The workflow proposed in this thesis is explicitly designed to keep a human in the loop and to make the basis for LLM suggestions transparent. First, mapping regeneration outputs concrete, readable Python modules (e.g. `etl/transform_generated.py`) that are

grounded in exposed artefacts: schemas, change sets, and reference transforms. Second, every generated mapping is compared against the deterministic baseline using quantitative metrics such as cell-level agreement, null-mismatch counts, and row-count differences, which are recorded in machine-readable form and visualised in figures. These checks provide engineers with structured evidence about how close a candidate mapping is to the known-good baseline, rather than forcing them to rely on ad hoc spot checks.

Looking ahead to production integration (as outlined in the Outlook), the design assumes explicit approval and rollback mechanisms for generated transformations. New mappings would be surfaced as proposals that must be reviewed and approved before being promoted to a live pipeline, and the previously validated transform and its metrics would be retained to enable rapid rollback if monitoring detects regressions. This combination of evidence-grounded prompts, explicit comparisons to a deterministic baseline, and human approval/rollback is intended to mitigate automation bias: the LLM acts as a powerful assistant for generating and adapting mappings, but responsibility for deciding whether to adopt a particular transform remains with the human operators, supported by quantitative and qualitative diagnostics.

6.6 Limitations and Threats to Validity

Several limitations constrain the extent to which the reported results generalise beyond the specific scenarios studied here.

6.6.1 Dataset and Scenario Scope

First, all experiments are conducted on a single real-world inspired domain: NYC taxi trip records. Although the constructed V1, V2, and canonical schemas capture common change patterns such as additions, removals, renames, and type changes, they do not span the diversity of enterprise data environments, including denormalised warehouse schemas, deeply nested records, or cross-table refactorings. V2 $\rightarrow$ V3 evolution is supported in the codebase but less thoroughly evaluated than V1 $\rightarrow$ V2 and canonical $\leftrightarrow$ V2. As a result, external validity to other domains and richer evolution histories is limited.

6.6.2 Model and Prompt Dependence

Second, the mapping regenerator is evaluated on a small set of LLMs and a specific prompt design that encodes schemas, changes, and reference transforms. Different models, decoding parameters, or prompt formats may yield substantially different behaviours. The stark difference between the `llama3` and `mistral` canonical ingestion results illustrates this sensitivity. Moreover, the experiments assume that reasonably clean reference transformation logic is available; in legacy ETL systems with fragmented or poorly documented code, assembling suitable prompts may be more challenging.

6.6.3 Operational and Organisational Factors

Third, the evaluation focuses on short- to medium-term adaptation effort in a controlled development environment, rather than long-term operational aspects such as cost, latency under load, governance, and team adoption. API pricing, data-residency constraints, and engineer trust in AI-generated code are only discussed qualitatively. These factors can significantly influence whether organisations are willing to deploy LLM-based mapping regeneration in production, and systematic study of them is left to future work.

6.7 Relation to Commercial Tools and Practical Implications

Commercial data platforms and ETL tools provide important context for interpreting the contributions of this thesis. Cloud data warehouses such as Snowflake, BigQuery, and Databricks offer schema-on-read and schema evolution features that make ingestion robust to many upstream changes, and ETL frameworks such as dbt, Fivetran, and AWS Glue provide extensive support for versioning, monitoring, and deployment. However, these systems typically stop short of automatically regenerating custom transformation logic in response to schema evolution.

For example, dbt models can be versioned and materialised incrementally, and dbt snapshots can track slowly changing dimensions, but adapting a model to a renamed or split column still requires manual edits to SQL code. Fivetran and similar ELT connectors can automatically add new source fields and adjust destination schemas but generally do not

infer or rewrite downstream business logic such as the taxi `trip_revenue` calculation or outlier filters. AWS Glue and related schema registries can validate compatibility between producers and consumers but rely on engineers to update transformation jobs when compatibility breaks.

In contrast, the workflow implemented in this thesis focuses specifically on using retrieved schema and change artefacts plus existing transformation code to *propose* updated mappings automatically. Rather than competing with commercial tools on orchestration or governance, it complements them at the transformation layer: schema registries, snapshots, and connector metadata can serve as rich sources of non-parametric knowledge for the RAG component, while the generated mappings can be surfaced to engineers for review and integration into existing pipelines. The empirical results indicate that such AI-assisted regeneration can be both accurate (for suitable models) and time-saving in controlled scenarios, suggesting a promising direction for practitioner tooling.

6.8 Summary

Overall, the discussion shows that the proposed workflow can detect designed schema changes exactly in the evaluated canonical scenario, regenerate mappings that match a strong deterministic baseline for at least one LLM model, and reduce adaptation effort relative to purely manual workflows in a controlled taxi domain. At the same time, the results expose sensitivity to model choice, limitations in removal detection and data-quality robustness, and narrow domain coverage. These findings motivate the future work outlined in the concluding chapter, particularly around broader empirical evaluation, richer change taxonomies, and tighter integration with commercial ETL platforms and monitoring systems.

Chapter 7

Conclusions

7.1 Conclusions

7.1.1 Summary of Contributions

This thesis investigated how retrieval-augmented large language models can support adaptive maintenance of ETL pipelines under schema evolution. Starting from the observation that even moderate structural changes in source schemas can silently break downstream transformations, the work proposed an integrated workflow that (i) detects and classifies schema evolution events, (ii) represents schemas and changes in machine-readable JSON artefacts, and (iii) leverages an LLM-based mapping regenerator to produce updated transformation code grounded in these artefacts and in existing business logic. The implementation is realised in a Python codebase that operates on both synthetic and real-world inspired datasets, with particular emphasis on a controlled evolution of the NYC yellow taxi schema.

Conceptually, the thesis contributes a concrete instantiation of evidence-grounded retrieval-augmented generation for schema-centric ETL tasks. Rather than treating the LLM as an opaque code generator, the approach constructs a retrieval corpus from historical schema versions, machine-readable change artefacts, and previously validated transformation code, and composes these artefacts with the model to encourage semantic preservation. Practically, the work delivers a reusable schema change detection module, an AI mapping regenerator, and executable ETL pipelines that together form a testbed for studying AI-assisted adaptation workflows. The overall design and implementation bridge ideas from

classical schema evolution and mapping adaptation [19, 20, 16] with recent advances in LLM-based data engineering [3, 11, 15, 2].

7.1.2 Answers to the Research Questions

The empirical results presented in Chapter 5 and interpreted in Chapter 6 allow us to answer the research questions at a high level. For **RQ1**, the schema change detector is able to recover the designed evolution events exactly in the canonical–V2 scenario, providing a reliable front-end for downstream regeneration under the specific flat tabular schemas considered, while highlighting that some change types (such as removals in other scenarios) remain more challenging. For **RQ2**, the mapping regenerator can, in favourable cases, produce canonical ingestion mappings that match a strong deterministic baseline exactly (as demonstrated for `llama3` on NYC taxi V2), while other models exhibit concrete failure modes, underscoring the importance of model choice, prompt design, and validation. For **RQ3**, the controlled experiments show that the AI-assisted workflow reduces time-to-adaptation relative to a purely manual workflow for the evaluated taxi scenario, without degrading basic data-quality checks, although human review remains essential for production use.

7.1.3 Outlook and Future Work

The thesis opens several directions for future research and development. A first avenue is to scale the evaluation to larger and more heterogeneous corpora of schema histories, including open data portals and versioned enterprise warehouses. This would enable more robust measurements of detection accuracy and adaptation effectiveness, and would allow benchmarking against alternative mapping adaptation tools and industrial ETL platforms.

A second direction is to refine the interaction between schema detection, retrieval, and generation. For example, richer change taxonomies and more sophisticated rename detection could capture complex structural refactorings, while retrieval strategies could be extended to include historical pipeline runs, log messages, or domain documentation.

On the generation side, incorporating automated test case synthesis or property-based checks could further reduce the risk of subtle semantic regressions.

Finally, integrating the proposed workflow into production-grade orchestration systems such as Apache Airflow or cloud-native data platforms would make it possible to study continuous adaptation under real operational constraints. This includes exploring human–AI collaboration patterns, designing approval and rollback mechanisms for generated transformations, and investigating how feedback from pipeline monitoring can be fed back into the LLM prompting and retrieval components. Taken together, these directions point towards a future in which ETL pipelines become more resilient and self-adapting, with generative AI acting as a transparent, controllable partner rather than a black-box replacement for human data engineers.

References

[1] M. I. L. Balaka, D. Alexander, Q. Wang, Y. Gong, A. Krisnadhi, and R. C. Fernandez, "Pneuma: Leveraging LLMs for tabular data representation and retrieval in an end-to-end system," *arXiv preprint arXiv:2504.09207*, 2025, SIGMOD 2025 / CIDR 2026 (cit. on p. 17).

[2] C. Buss, M. Safari, A. Termehchy, S. Lee, and D. Maier, "Towards scalable schema mapping using large language models (LLMs)," *arXiv preprint arXiv:2505.24716*, 2025. DOI: `10.48550/arXiv.2505.24716` (cit. on pp. 49, 56).

[3] M. Chen et al., "Evaluating large language models (LLMs) trained on code," *arXiv preprint arXiv:2107.03374*, 2021 (cit. on pp. 1, 4, 9, 17, 51, 56).

[4] C. Curino, H. J. Moon, and C. Zaniolo, "Schema evolution in Wikipedia: Toward a web information system benchmark," in *Proceedings of the 10th International Conference on Enterprise Information Systems (ICEIS)*, 2008, pp. 323–332 (cit. on pp. 13, 47).

[5] M. Di Profio, M. Zhong, Y. Sripada, and M. Jaspars, "FlowETL: An autonomous example-driven pipeline for data engineering," *arXiv preprint arXiv:2507.23118*, 2025 (cit. on pp. 18, 51).

[6] A. Doan and A. Y. Halevy, "Semantic integration research in the database community: A brief survey," *AI Magazine*, vol. 26, no. 1, pp. 83–83, Mar. 2005 (cit. on p. 7).

[7] A. Frenk and R. Shraga, "SAVeD: Semantic aware version discovery," *arXiv preprint arXiv:2511.17298*, 2025 (cit. on p. 19).

[8] O. E. Gungor, D. Paulsen, and W. Kang, "Schemora: Schema matching via multi-stage recommendation and metadata enrichment using off-the-shelf LLMs," *arXiv preprint arXiv:2507.14376*, 2025 (cit. on p. 15).

[9] M. Kayali, F. Wenz, N. Tatbul, and Ç. Demiralp, "Mind the data gap: Bridging LLMs to enterprise data integration," *arXiv preprint arXiv:2412.20331*, 2024 (cit. on p. 49).

[10] K. S. Kumar et al., "AutoSchema: Self-supervised learning framework for schema drift detection and adaptation," *European Journal of Advances in Engineering and Technology*, vol. 10, no. 7, pp. 94–100, 2024 (cit. on p. 19).

[11] P. Lewis et al., "Retrieval-augmented generation (RAG) for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474 (cit. on pp. 2, 3, 9, 17, 49, 56).

[12] X. Li et al., "Llm-match: An open-sourced patient matching model based on large language models and retrieval-augmented generation," *arXiv preprint arXiv:2503.13281*, 2025 (cit. on p. 15).

[13] Y. Liu, E. Pena, A. Santos, E. Wu, and J. Freire, "Magneto: Combining small and large language models for schema matching," *Proceedings of the VLDB Endowment*, vol. 18, 2024, arXiv:2412.08194 (cit. on p. 15).

[14] C. Ma, S. Chakrabarti, A. Khan, and B. Molnár, "Knowledge graph-based retrieval-augmented generation (RAG) for schema matching," *arXiv preprint arXiv:2501.08686*, 2025 (cit. on p. 16).

[15] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022 (cit. on pp. 2, 18, 49, 51, 56).

[16] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," *The VLDB Journal*, vol. 10, no. 4, pp. 334–350, 2001 (cit. on pp. 1, 13, 15, 56).

[17] N. Seedat and M. van der Schaar, "Matchmaker: Schema matching with self-improving compositional LLM programs," *arXiv preprint arXiv:2410.24105*, 2024, Accepted to ICML 2025 (cit. on p. 16).

[18] S. Tera, "Autonomous ETL pipelines: Using generative AI to design, validate, and deploy dataflows," *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, vol. 6, no. 4, pp. 204–211, 2025 (cit. on pp. 18, 51).

[19] Y. Velegrakis, R. J. Miller, and L. Popa, "Mapping adaptation under evolving schemas," in *Proceedings of the 29th International Conference on Very Large Data Bases (VLDB)*, Berlin, Germany, 2003 (cit. on pp. 1, 2, 4, 8, 12, 47, 48, 56).

[20] C. Yu and L. Popa, "Semantic adaptation of schema mappings when schemas evolve," in *Proceedings of the 31st International Conference on Very Large Data Bases (VLDB)*, Trondheim, Norway, 2005, pp. 1006–1017 (cit. on pp. 1, 2, 4, 8, 13, 47, 48, 56).

[21] T. Zhang et al., "EvoSchema: Evaluating text-to-SQL robustness against schema evolution," *Proceedings of the VLDB Endowment*, vol. 18, pp. 3655–3668, 2025 (cit. on p. 19).